

CHAPTER 3: MongoDB Architecture

"MongoDB architecture covers the deep-dive architectural concepts of MongoDB."

Core Processes: The core components in the MongoDB package are,

- mongod, which is the core database process.
- mongos, which is the controller and query router for sharded clusters.
- mongo, which is the interactive MongoDB shell.

MongoDB Tools:

- Apart from the core services, there are various tools that are available as part of the MongoDB installation:

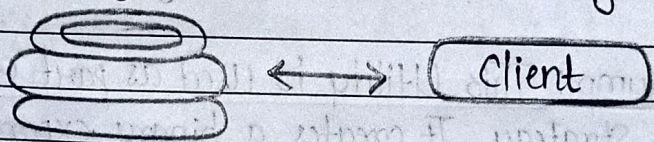
- mongodump: This utility is used as part of an effective backup strategy. It creates a binary export of the database contents.
- mongorestore: The binary database dump created by the mongodump utility is important to a new or an existing database using the mongorestore utility.
- bsondump: This utility converts the BSON files into human-readable formats such as JSON and CSV. For example, this utility can be used to read the output file generated by mongodump.
- mongoimport, mongoexport: mongoimport provides a method for taking data in JSON, CSV, or TSV formats and

importing it into a mongod instance. `mongoexport` provides a method to export data from a mongod instance into JSON, CSV, or TSV formats.

- `mongostat`, `mongotop`, `mongosniff`: These utilities provided diagnostic information release related to the current operation of a mongod instance.

Standalone Deployment:

- Standalone Deployment is used for development purpose; it doesn't ensure any redundancy of data and it doesn't ensure recovery in case of failure.
- So it's not recommended for use in production environment. Standalone deployment has the following components: a single mongod and a client connecting to the mongod,



IMP ✓

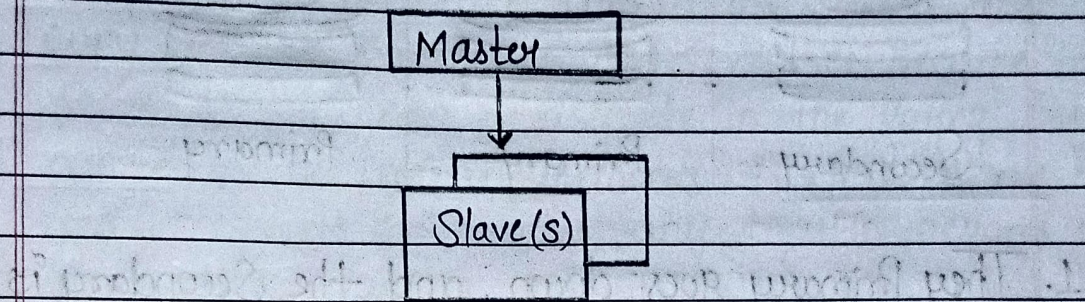
Replication: It provides for data redundancy by a replicating data on different nodes, thereby providing protection of data in case of node failure, replication provides high availability in a MongoDB deployment.

- There are two types of replication supported in MongoDB: traditional master/slave replication and replica set.

(a) Master/Slave Replication:

- In MongoDB, the traditional Master/Slave replication is available but it is recommended only for more than 50 node replications, the preferred replication approach is

Replica Sets. In this type of replication, there is one master and number of Slaves that replicate the data from the master. The only advantages with type of the replication is that there's no restriction on the number of slaves within a cluster. However, thousands of slaves will burden over the master node, so in practical scenarios it's better than to have less than dozen slaves.

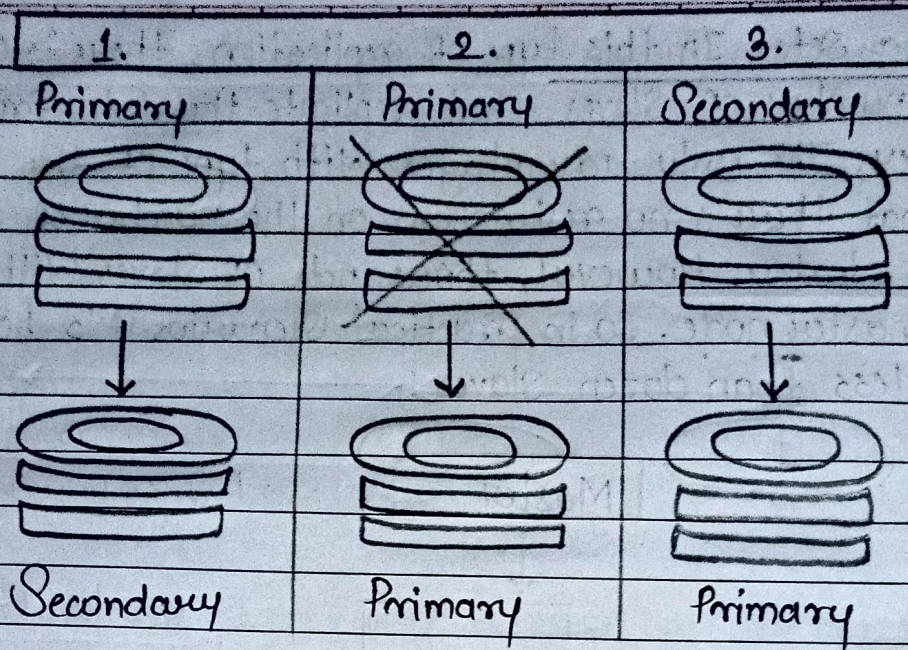


→ There are two main reasons behind a Slave becoming Out of Sync:

- (1) The Slave shuts down or stops and restarts later. During this time, the Oplog may have deleted the log of operations required to be applied on the slave.
- (2) The Slave is slow in executing the updates that are available from the master.

(b) Replica Set:

→ Replica sets are basically a type of master-slave replication but they provide automatic failover. A replica set has one master, which is termed as primary, and multiple slaves, which are termed as secondary in the replica set context; however, unlike master-slave replication, there's no node of one that is fixed to be primary in the replica set.



1. The Primary goes down, and the Secondary is promoted as primary.
2. The Original Primary comes up, it acts as slave, and becomes the Secondary node.

Primary and Secondary Members:

(1) Primary Member: A replica set can have only one primary, which is elected by the voting nodes in the replica set. Any node with associated priority as 1 can be elected as a primary.

(2) Secondary Member: A normal secondary member holds the copy of the data. The secondary member can vote and also can be a candidate for being promoted to primary in case of failover of the current primary.

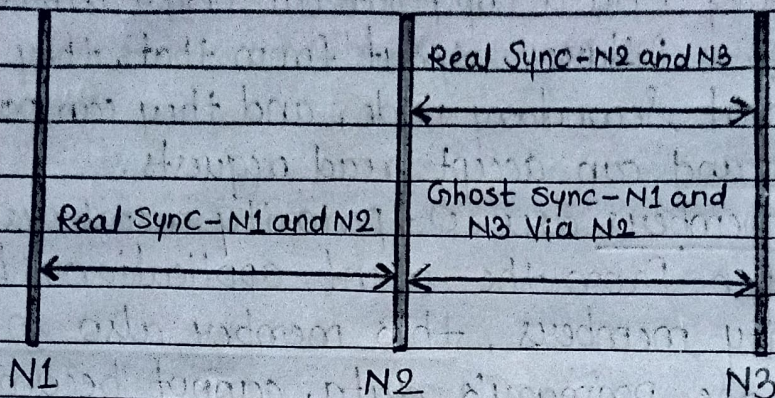
Types of Secondary Members:

- (1) Priority 0 members are Secondary members that maintain the primary's data copy but can never become a Primary in case of a failover. Apart from that, they function as a normal Secondary node, and they can participate in voting and can accept read requests.
- (2) Hidden members are 0-priority members that are hidden from the client applications. Like the 0-priority members, this member also maintains a copy of the primary's data, cannot become the primary, and can participate in the voting, but like or Unlike 0-priority members, it can't serve any read requests or receive any traffic beyond what replication requires.
- (3) Delayed members are Secondary members that replicate data with a delay from the primary's Oplog. This helps to recover from human errors, such as accidentally dropped databases or errors that were caused by unsuccessful application upgrades.
- (4) Arbiters are Secondary members that do not hold a copy of the primary's data, so they can never become the primary. They are solely used as members for participating in voting. This enables the replica set to have an uneven number of nodes without incurring any replication cost which arises with data replication.
- (5) Non-voting members hold the primary's data copy, they can accept client read operations, and they can also become the primary, but they can't voice to vote in an election.

This topic has to be

Election → forcing a New Election {Self-Studied}

Making Writes Work with chaining Slaves:



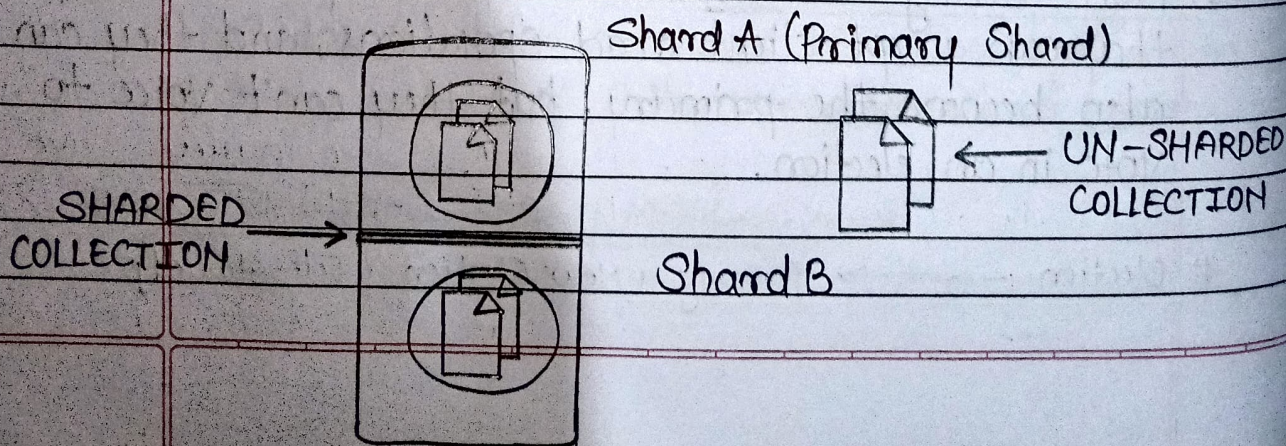
Sharding:

(1) Sharding Components:

The following are the components of a Sharded cluster:

- Shards
- mongos
- config Servers

The Shards is the component where the actual data is stored. For the Sharded cluster, it add holds a subset of data and can either be a removed mongod or a replica set. All Shard's data combined together forms the complete dataset for the Sharded cluster.



Date _____
Page _____

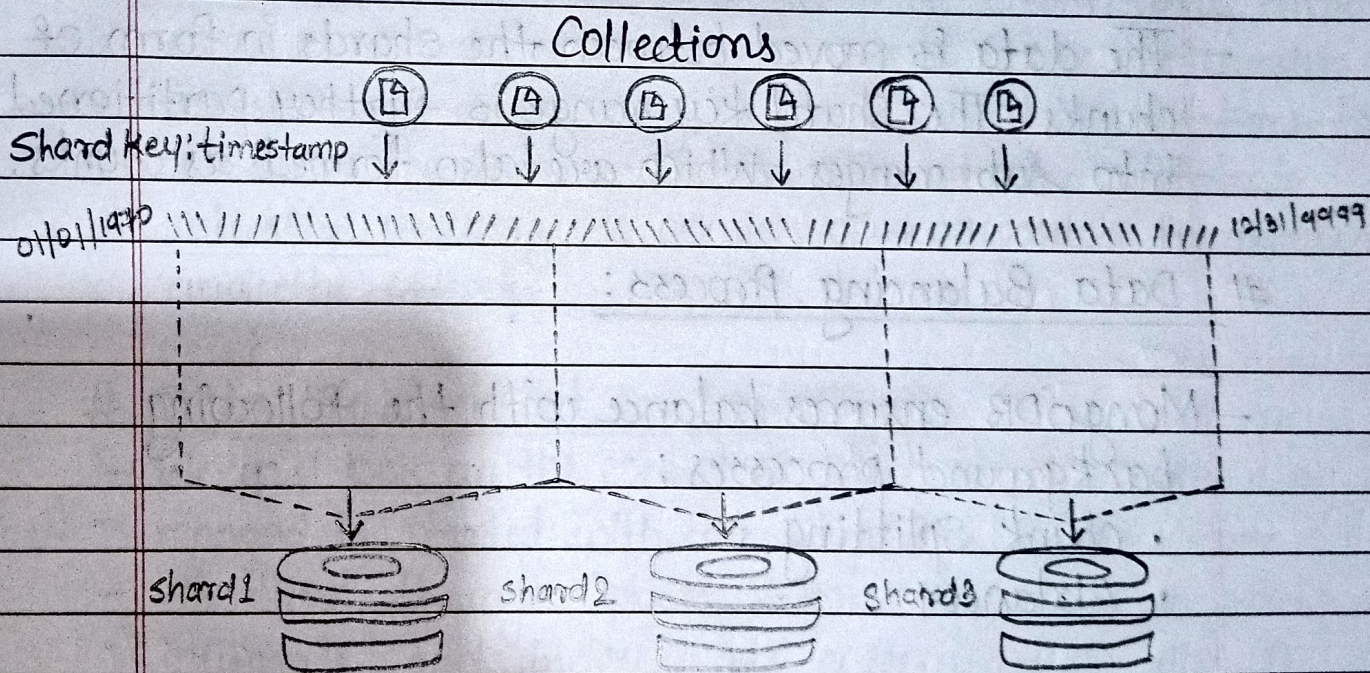
- Data Distribution Process:

- In Mongo, the data is sharded or distributed at the collection level. The collection is partitioned by the key was shard key.

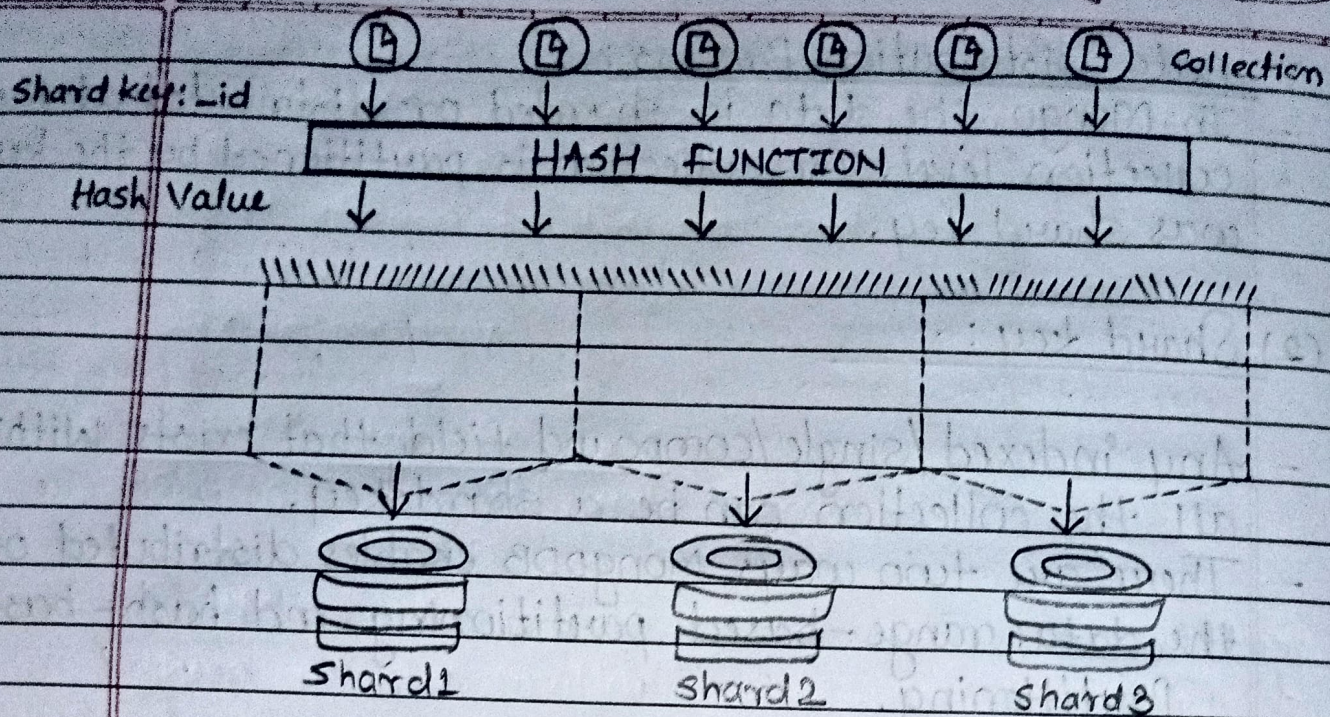
(2) Shard key:

- Any indexed single/compound field that exists within all the collection can be a shard key.
- There are two ways MongoDB enables distributed of the data: range-based partitioning and hash-based partitioning.

(a) Range-based Partitioning:



(b) Hash-Based Partitioning:



Chunks:

- The data is moved between the shards in form of chunks. The shard key range is further partitioned into sub-ranges, which are also termed as chunks.

Data Balancing Process:

- MongoDB ensures balance with the following background processes:
 - chunk splitting
 - Balancer

CHAPTER 1: MongoDB Explained

"MongoDB explained covers deep-dive concepts of what happens under the hood in MongoDB."

- We will learn how data is stored in MongoDB and how writes happen using journaling. We will learn about GridFS and the different types of indexes available in the MongoDB.

Data Storage Engine:

- MongoDB Version 3.0 ships with two storage engines.

Data File (Relevant for MMAPv1):

- Extent: Extent refers to a group of data records within a data file, so a group of extents forms the complete data for a namespace. An extent uses disk locations to refer to the location on the disk where the data is actually residing. It consists of two parts: file numbers and the offset.

Data File (Relevant for WiredTiger)

- We will look at the content of data, directory when the mongod is started with the WiredTiger storage engine.
- When the storage option selected is WiredTiger, data, journals, and indexes are compressed on disk.

IMP # How Data Is Written Using Journaling: refer to the book

IMP # GridFS - The MongoDB file system:

- GridFS is MongoDB's specification for handling large files.

that exceeds BSON's document Size limit.

- The GridFS Specification is useful in cases where the file size exceeds the default 16MB limitation of MongoDB BSON document. It's also used for storing files that you will need to access without loading the entire file in memory.

GridFS under the Hood:

- GridFS is a lightweight Specification used for storing files.
- GridFS enables you to store large files by splitting them up into smaller chunks and storing each of the chunks as separate documents. In addition to these chunks, there's one more document that contains the metadata about the file. Using this metadata information, the chunks are grouped together, forming the complete file.
- The Storage Overhead for the chunks can be kept to a minimum, as MongoDB supports storing binary data in the documents.

Ex.

```
{  
  "_id": ObjectId("..."), "n": 0, "data": BinData("..."),  
  "files_id": ObjectId("...")  
}
```

Chap 10 (in Book) → Self Study
Unit-3 (Chap 2) MongoDB Use cases